COS314 ASSIGNMENT 3

Dr Thambo Nyathi

MAY 18, 2026

Ashiv Ramballi: u26854857
Okaile Gaesale: u23527685

Table of Contents

1. Genetic Programming Parameters

Parameter	Arithmetic GP Value	Logical GP (Decision Tree) Value
Population size	200	200
Maximum generations	100	100
Initial tree generation	Ramped half-and-half	Ramped half-and-half
Initial tree depth	3	2-7
Max offspring depth	5	5
Selection method	Tournament Selection	Tournament Selection
Tournament size	15	7

Function set	`+`, `-`, `*`, protected `/`	`<`, `>`, `<=`, `>=`
Crossover rate	85%	70%
Mutation rate	15%	30%
Mutation type	Point Mutation	Point Mutation
Mutation offspring depth	7	7
Fitness function	Classification Accuracy	60% F1 + 40% Balanced Accuracy (with parsimony pressure)

2.Model descriptions

Symbolic approach using arithmetic GP classifiers:

In the Arithmetic GP model, each candidate solution is represented by an arithmetic expression tree. Every internal node in the tree is an arithmetic operator, and every leaf node is either a constant terminal or a feature terminal.

The terminal set consists of the nine Breast Cancer dataset features (represented by `x0` to `x8`), as well as randomly generated integer constants between 0 and 10. The feature mapping is as follows:

- X0 = age
- X1= menopause
- X2= tumor_size
- X3= inv_nodes
- X4= node caps
- X5= degree_malignancy
- X6= breast
- X7+ breast_quad
- X8= irradiat

The function set includes addition, subtraction, multiplication, and protected division. Protected division is utilized to prevent runtime crashes; if the denominator is zero (or exceptionally close to zero), the function returns a safe default value (1.0) rather than throwing an exception.

During evaluation, each row in the training dataset is passed through the arithmetic tree. The resulting numerical output is converted into a class prediction using a zero-threshold criterion: if the output is ≥ 0, the model predicts class 1 (recurrence-events); otherwise, it predicts class 0 (no-recurrence-events). The fitness of the arithmetic GP is evaluated purely on its classification accuracy across the training set. The GP utilizes point mutation, subtree crossover, and tournament

selection to evolve the population over 100 generations, saving the best overall individual as the final classifier

Logical approach using evolved decision trees:

In contrast to the Arithmetic GP, the Logical GP represents candidate solutions as logical rule trees (Decision Trees). Instead of generating a mathematical formula, it evaluates Boolean conditions to route data and produce a classification decision.

Every internal node in the logical tree contains a condition based on a dataset feature. These conditions use comparison operators (<, >, <=, >=) to compare a specific feature's value against a randomly generated threshold appropriate for that feature's bounds (e.g., `tumor_size >= 4`). The tree splits into two alternative routes ("yes" or "no") based on whether the criterion is met. This routing continues until a leaf node is reached, which contains the final predicted class label (0 or 1).

To combat overfitting and bias toward the majority class, the Logical GP uses a highly specialized fitness function. It evaluates individuals using a blend of 60% F-measure and 40% Balanced Accuracy. Additionally, it applies specific penalties for False Negatives (which carry higher medical risk) and False Positives, alongside a parsimony pressure penalty (based on tree size) to prevent uncontrolled tree growth (bloat). The model evolves using tournament selection, subtree crossover, and a point mutation strategy that randomizes node conditions without altering the tree's structural shape.

3.Pre-processing steps

General dataset pre-processing:

The Breast Cancer dataset was supplied in CSV format, split into separate training and testing files. The GP models were developed strictly using the training file, keeping the testing file hidden to act as unseen data for the final evaluation.

During ingestion, the header row of the CSV was omitted. Each row was split by commas, with the first column parsed as the target class label and the remaining nine columns stored in a numerical array as input features. Because the dataset utilizes integer mappings to express categorical variables (e.g., age, menopause, node-caps) and binary labels, the data was natively ready for processing. The program also features robust error handling to verify consistent column lengths across all rows, aborting on faulty data.

- CSV column 0 = class label / actual class
- CSV column 1 = age
- CSV column 2 = menopause
- CSV column 3 = tumor_size
- CSV column 3 = tumor_size
- CSV column 4 = inv_nodes
- CSV column 5 = node_caps
- CSV column 6 = degree_of_malignancy
- CSV column 7 = breast
- CSV column 8 = breast_quad
- CSV column 9 = irradiat

Arithmetic pre-processing:

The CSV file was loaded for the arithmetic GP model by first reading and omitting the header row. Commas were used to divide each remaining row. The class label was parsed from the first value in the row, and the other entries were parsed as numerical feature values. These feature values served as the terminals x0 through x8 and were kept in a double[] array. Before providing the dataset, the arithmetic

code also verifies that every row has the same number of feature columns. The program stores the class labels in an integer array and the feature values in a two-dimensional array after reading every row. Each evolving arithmetic tree may be assessed against the feature values and its prediction can be compared to the appropriate class label thanks to this separation.

Additionally, the program verifies that the number of feature columns in each row is the same. This keeps training from using inconsistent rows. The program returns an error rather than proceeding with faulty data if the CSV file is empty or if the number of columns is mismatched.

The arithmetic GP program did not use any feature scaling, normalization, standardization, or class balance. The values of the encoded numerical features were utilized exactly as they were found in the CSV file. This indicates that the GP developed expressions based on the dataset's current integer encodings.

The nine input features, denoted by x0 through x8, made up the arithmetic GP's terminal set. The GP was permitted to produce random integer constants between 0 and 10 in addition to these feature terminals. These constants were created by the GP system and utilized as potential terminal values in expression trees rather than being extracted directly from the dataset.
The arithmetic GP therefore uses the dataset in the following format:

- label, x0, x1, x2, x3, x4, x5, x6, x7, x8

the functional set consisted of:

- +,-,*, protected division(/)

Because regular division can fail when the denominator is zero, protected division was included. Instead of crashing during evaluation, the software returned a safe value if the denominator was zero or very near to zero. This made it possible to successfully evaluate each generated arithmetic expression on each row of the dataset.

The actual class label from column 0 was compared to the predicted. The training accuracy, which functioned as the fitness function for the arithmetic GP, was then determined by counting the number of accurate predictions.

Explicit feature conditions like age > 3 or tumor_size <= 4 were not manually created by the arithmetic GP. Instead, it allowed these linkages to arise through evolved arithmetic expressions. For instance, an equation like (x5 / (x5 - 2)) can establish a nonlinear relationship including degree_of_malignancy, but an expression like (x3 - 4) can indirectly reflect a comparison involving the inv_nodes feature. This made it possible for the model to look for mathematical combinations of the encoded data

that could be able to distinguish between the two groups.

Logical pre-processing:

The pre-processed breast cancer CSV files utilized for the arithmetic model were also used for the logical genetic programming model. Prior to being entered into the application, the dataset had already been transformed into numerical form. This was required since the logical model compares feature values to numerical thresholds in order to generate decision rules. The assignment dataset employs integer mappings to express categorical variables such as age, menopause, node-caps, breast, breast quadrant, and irradiat, as well as encoded class labels and encoded categorical feature values, such as 0 = no-recurrence-events and 1 = recurrence-events.

The CSV file's first row was not used as training data; instead, it was handled as the header row. One patient record was represented by each of the following rows. Each row's first column served as the actual class label, and the other columns served as the logical decision tree's input features.

The logical GP did not make direct use of the properties found in mathematical expressions, in contrast to the arithmetic GP. Rather, Boolean decision criteria were constructed using the encoded numerical values. Every internal node in the logical tree used a comparison operator to compare one characteristic to a threshold value. The model might, for instance, provide circumstances like:

- x2 >= 4
- x3 < 3
- x5 >= 2
- x8 <= 1

The categorized and binned dataset values had already been transformed into integers, making these conditions feasible. The logical GP would not be able to apply operators like <, >, <=, and >= to the original text-based categories without this encoding step.

There was no further scaling or normalization done during pre-processing. This indicates that the feature values were utilized by the logical model precisely as they were in the encoded CSV files. Because the logical GP does not rely on gradient-based learning or distance calculations, this was appropriate. Rather, it looks through the current encoded values for practical threshold-based rules.

During pre-processing, no class balancing was used. Any imbalance between the no-recurrence-events and recurrence-events classes persisted in the training data since the training set was used exactly as supplied. This is significant because the logical GP's evolved decision rules may be impacted by class imbalance. Evaluation utilizing both accuracy and F-measure was required because a model trained on unbalanced data may acquire rules that favour the majority class. Additionally, feature selection was not done by the logical GP before to training. The model was given access to all nine dataset features. By developing decision trees over several generations, this enabled the GP process to determine which traits were beneficial. A feature may show up in internal decision nodes if it was helpful for categorization. Evolution might naturally result in trees that employed a trait less frequently or not at all if it was not beneficial.

The logical GP created decision nodes using the feature values when the data was loaded. Every decision node was made up of:

- A selected feature
- A comparison operator
- A threshold value

The value of x5 was extracted from the patient record and compared to the threshold during the evaluation process. The tree followed one branch if the criterion was met. The tree followed the other branch if the requirement was not met. Until a leaf node was reached, this process was repeated. The expected class was present in the leaf node.

As a result, the pre-processed numerical information was transformed into a rule-based classification issue by the logical GP. Every path through the tree constituted a series of logical tests on the patient's encoded feature values, and each developed tree represented a potential decision-making structure.

The logical GP was trained and tested using the same split as the arithmetic GP. The testing CSV was hidden until the last stage of classification, whereas the training CSV was utilized to develop the decision trees. This made it possible to objectively assess the logical GP using data that wasn't used throughout evolution. Using the identical pre-processed files made it possible to compare the two models under the same circumstances because the assignment required both GP techniques to report training accuracy, test accuracy, and F-measure.

Final results

Algorithm	Train(%)	Test(%)	F-Measure	Runtime(s)
Logical GP training accuracy	75,96%	72,09%	0,7143	0,13
Arithmetic GP training accuracy	83,61%	65,12%	0,5213	0.05

The above are the results that were produced by the best run for each of the algorithms. For the logical tree, it was run number 25 which used a seed of 56982693090 and for the arithmetic tree, it was run number 22 and seed of 500.

Wilcoxon test

The Wilcoxon test is based on the runs for the logical and arithmetic trees that produced the best results. For the logical tree, it was run number 25 which used a seed of 56982693090 and for the arithmetic tree, it was run number 22 and seed of 500. The Wilcoxon test will take into account the testing accuracy between the two models for 5 of the 100 generations for their best seeded run:

Generation	Arithmetic GP training accuracy	Logical GP training accuracy	Difference
1	79,78	74,86	4,92
25	83,61	75,96	7.65
50	83,61	75,96	7.65
75	83,61	75,96	7.65
100	83,61	75,96	7.65

the difference calculation was represented by the following formula:

- Difference = Arithmetic GP accuracy - Logical GP accuracy

All differences are positive, meaning that the arithmetic GP had the higher training accuracy for all selected generations

The sum of the ranks of all of the positive differences (W+): 15

The sum of the ranks for all of the negative differences(W-): 0

Wilcoxon test statistic (W): 0

Two tailed exact p-value: 0,0625

The above p value tells us whether the observed difference is statistically significant so the possible smallest two-tailed exact p value when all differences go in the same direction is 0,0625